

FAST-MVH: Fast Adaptive Search with Trees for Multi-Valued Heuristic Search

Abstract

Multi-valued heuristics describe trade-offs that a single lower-bound vector cannot represent, but selecting a heuristic may require many dominance checks. FAST-MVH reduces this cost while preserving the search decisions of $L\text{-NAMOA}_{dr}^*\text{-mvh}$. It indexes expensive frontiers with adaptive k-d trees, tests local dominance before selecting a heuristic, and reuses dominance witnesses. Exact full-dimensional checks remain available when dimensionality reduction alone cannot decide a query. Experiments cover grids and the New York and Bay Area road networks with three to six objectives. On a new six-objective grid, FAST-MVH returns the same 70,196 solution costs in 2.67 s rather than 102.85 s. Three-objective gains are smaller and depend on frontier size. Cache ablations show both improvements and regressions: witness reuse helps only when saved queries repay its overhead. We give the algorithm, establish decision equivalence, and separate complete baseline comparisons from time-limited runs.

1 Introduction

A shortest path need not be best in every objective. One route may be shorter while another is faster. Multi-objective search must therefore retain paths with incomparable costs, rather than one cheapest path per state. As the retained sets grow, testing whether another path is dominated can cost more than generating it.

Multi-valued heuristics (MVHs) address a different difficulty: weak lower bounds. Instead of one estimate, an MVH gives several estimates of the remaining cost. Each describes a possible trade-off. This additional information can reduce search, but using it is not free. A path may test several heuristic vectors before finding one that the current solutions do not refute.

Wolff, Felner, and Salzman [1] combine MVHs with dimensionality reduction in $L\text{-NAMOA}_{dr}^*\text{-mvh}$. Each path has one active heuristic vector. If a newly found solution refutes that vector, the algorithm selects another and reinserts the path. Local dominance checks use truncated vectors when justified and full vectors otherwise. This lazy treatment avoids generating a separate search node for every heuristic.

FAST-MVH retains the $L\text{-NAMOA}_{dr}^*\text{-mvh}$ algorithm [1]. Its contribution is to reduce the work needed to make the same decisions. It changes the representation of frontiers, the order of heuristic selection and local dominance checks at

generation, and the reuse of positive dominance answers. It does not change the heuristic, the ordering of OPEN, or the Pareto set being sought. The dynamic k-d tree indexing of frontiers follows prior work on geometric indexing [2]; the contribution here is its integration with lazy MVH evaluation and witness caching.

2 Problem and Search Invariants

Let $G = (S, E, c)$ be a finite graph with $c(e) \in \mathbb{R}_{\geq 0}^M$. Path costs are component-wise sums. A vector x weakly dominates y , written $x \preceq y$, if $x_k \leq y_k$ for every k . It dominates y if also $x \neq y$. The task is to return one path for each distinct non-dominated start-to-goal cost vector. Write $x \prec_{\text{lex}} y$ for lexicographic order and

$$\text{Tr}(x) = (x_2, \dots, x_M).$$

An admissible MVH assigns a finite set $H(s)$ to each state: for every goal-reaching suffix π from s , some $h \in H(s)$ satisfies $h \preceq c(\pi)$. We assume nonempty heuristic sets on states searched, $H(s_{\text{goal}}) = \{0\}$, and the ordering used by [1]:

$$H(s) = \langle h^1, \dots, h^{k(s)} \rangle, \quad h^1 \leq_{\text{lex}} \dots \leq_{\text{lex}} h^{k(s)}.$$

For genuine MVHs this order is a correctness precondition, not a traversal preference. The selected vector must be the first surviving member. Sorting a spatial tree by another objective must not change that answer.

Example. Suppose two suffixes cost (1, 10) and (10, 1). The set containing these vectors is admissible, but its component-wise maximum (10, 10) bounds neither suffix. In contrast, if every vector separately bounds *every* suffix, their maximum is admissible. Landmark lower bounds have this universal property; genuine MVHs generally do not [3,4]. We do not replace an MVH by its maximum.

A node n records a state $s(n)$, path cost $g(n)$, heuristic index $i(n)$, and $f(n) = g(n) + h^{i(n)}$. OPEN is ordered lexicographically by f . At each state, $\mathcal{C}(s)$ stores expanded path costs. A reduced frontier $F(s) \subseteq \mathcal{C}(s)$ retains full vectors but is queried on their truncations. An expansion with cost g performs

$$F(s) \leftarrow \{p \in F(s) : \text{Tr}(g) \not\preceq \text{Tr}(p)\} \cup \{g\}.$$

Let $T = F(s_{\text{goal}})$ and $m(s) = \max_{p \in F(s)} p_1$, with $m(s) = 0$ when empty. Importantly, $F(s)$ is not assumed to be a full-dimensional Pareto set, or even an antichain after truncation. Its required property is coverage: every $a \in \mathcal{C}(s)$ has a representative $p \in F(s)$ with $\text{Tr}(p) \preceq \text{Tr}(a)$.

3 FAST-MVH

Algorithm 1 follows the node lifecycle of [1], which implements $L\text{-NAMOA}_{dr}^*\text{-mvh}$: extract, possibly reevaluate the heuristic, locally test, expand, and generate successors. The operative change is that at successor generation, the local test is moved to **after** heuristic selection, pruning dominated costs before creating a node. The parent of a reinserted node is unchanged. Equal-key queue ties and successor enumeration use the reference implementation's conventions.

Algorithm 1: FAST-MVH

Input: $G, s_{\text{start}}, s_{\text{goal}}$ and ordered admissible H .

Output: a cost-unique Pareto-optimal path set Sols.

```

1  Sols  $\leftarrow \emptyset$ ;  $C(s), F(s) \leftarrow \emptyset$  for all  $s$ 
2   $n \leftarrow (s_{\text{start}}, 0, 1)$ ;  $\text{parent}(n) \leftarrow \perp$ 
3  OPEN  $\leftarrow \{n\}$ 
4  while OPEN  $\neq \emptyset$  do
5     $n \leftarrow \text{OPEN.PopMin}()$ 
6    if GoalDom( $f(n)$ ) then
7       $i \leftarrow \text{ChooseH}(s(n), g(n), i(n) + 1, \text{true})$ 
8      if  $i \neq \perp$  then set  $i(n) \leftarrow i$ ; reinsert  $n$ 
9      continue
10   if LocalDom( $s(n), g(n)$ ) then continue
11   update  $F(s(n))$  with  $g(n)$ ; append  $g(n)$  to  $C(s(n))$ 
12   if  $s(n) = s_{\text{goal}}$  then
13     add  $n$  to Sols; continue
14   for each  $s' \in \text{Succ}(s(n))$  do
15      $g' \leftarrow g(n) + c(s(n), s')$ 
16      $i' \leftarrow \text{ChooseH}(s', g', 1, \text{false})$ 
17     if  $i' \neq \perp$  and  $\neg \text{LocalDom}(s', g')$  then
18       create  $n' = (s', g', i')$  with parent  $n$ 
19       insert  $n'$  in OPEN
20 return Sols

```

As in [1], the start uses h^1 . The archived implementation instead initializes its sole start node with $h = 0$. Since T and all closed sets are empty at its extraction, this initialization has the same subsequent search trajectory.

At successor generation (lines 16–19), heuristic selection precedes the local dominance test. This order ensures that ChooseH can fail early if no heuristic survives the goal frontier; if none exists, LocalDom is never called. On trade-off frontiers with many heuristics, this saves both selection and fallback costs. By contrast, the reference implementation's order (local test first) must query the frontier even for successors that will later fail selection.

3.1 Exact dominance queries

Let $\text{Exists}(F, q) \in \{\text{true}, \text{false}\}$ return true iff there exists $p \in F$ with $\text{Tr}(p) \preceq q$ (in the $M - 1$ truncated dimensions). Let

$m(s) = \max\{p_1 : p \in F(s)\}$ denote the greatest first coordinate in frontier $F(s)$, or 0 if $F(s) = \emptyset$.

The goal test (**GoalDom**) returns true iff a truncated goal bound $\text{Tr}(f)$ has a dominator in the target frontier T , pruning the node. The local test (**LocalDom**) returns true iff a path cost g is dominated by $C(s)$ in all M dimensions. Algorithm 2 optimizes these using a k-d tree index on $F(s)$'s truncated vectors and the exact maximum $m(s)$.

Algorithm 2: Dominance checks

```

1  function GoalDom( $f$ )
2    return Exists( $T, \text{Tr}(f)$ )
3  function LocalDom( $s, g$ )
4    if  $m(s) \leq g_1$  then return Exists( $F(s), \text{Tr}(g)$ ) using KD-tree query
5    found  $\leftarrow \text{false}$ 
6    for each  $p \in F(s)$  with  $\text{Tr}(p) \preceq \text{Tr}(g)$  do (via KD-tree scan)
7      found  $\leftarrow \text{true}$ 
8      if  $p_1 \leq g_1$  then return true
9    if not found then return false
10   return  $\exists a \in C(s) : a \preceq_M g$  (full-dimensional fallback scan)

```

3.2 Heuristic selection

A cache slot $W_s(i)$ stores the truncation of a goal vector that previously refuted candidate i . A second witness w is reused within a call. An undefined witness fails a comparison. The optional pointer $\rho_s(i)$ is the greatest earlier index whose heuristic truncation dominates $\text{Tr}(h^i)$, or zero if none is stored. Omitting such a pointer only loses a shortcut.

Algorithm 3: ChooseH(s, g, j, b)

Precondition: b is true only if candidate $j - 1$ was just refuted by the goal test.

```

1  if  $j > k(s)$  then return  $\perp$ 
2  if  $T = \emptyset$  then return  $j$ 
3   $r \leftarrow j - 1$  if  $b$ , and  $j$  otherwise;  $w \leftarrow \perp$ 
4  for  $i = j, \dots, k(s)$  do
5    if  $\rho_s(i) \geq r$  then continue
6     $q \leftarrow \text{Tr}(g) + \text{Tr}(h^i)$ 
7    if  $W_s(i) \preceq q$  or  $w \preceq q$  then continue
8    find  $t \in T$  with  $\text{Tr}(t) \preceq q$ , if one exists
9    if no such  $t$  exists then return  $i$ 
10    $W_s(i) \leftarrow \text{Tr}(t)$ ;  $w \leftarrow \text{Tr}(t)$ 
11 return  $\perp$ 

```

The cache stores vectors, not pointers into a mutable frontier. A miss always reaches an exact query. Candidate order is unchanged; a cache hit rejects a candidate but never accepts one.

4 Why the Changes Are Safe

Lemma 1 (Local exactness). Algorithm 2 returns true exactly when $\exists a \in C(s): a \leq g$.

Proof. Coverage of $C(s)$ by $F(s)$ holds initially. Removing a frontier vector preserves coverage because its replacement dominates its truncation. Thus a full dominator in $C(s)$ implies a truncated dominator in $F(s)$. If none exists, returning false is safe. When $m(s) \leq g_1$, every truncated dominator also dominates the first coordinate. A witness accepted at line 9 is likewise a member of $C(s)$ that dominates all coordinates. The remaining case is decided by the exact test at line 11. $\square$

Example. Let $C(s)$ contain $a = (3, 3, 3)$ and a later expansion $p = (9, 2, 2)$. Updating $F(s)$ removes a , since $\text{Tr}(p) \leq \text{Tr}(a)$. For a new path $g = (4, 3, 3)$, p is a truncated dominator but not a full one. The fallback is necessary: the removed vector a still dominates g . Storing only the reduced frontier would lose this information.

Lemma 2 (Persistent witnesses). If a vector once in T refutes a truncated query q , the current T also refutes q .

Proof. A goal vector is removed only when its replacement weakly dominates its truncation. Following replacements preserves domination of q by transitivity. $\square$

Consequently the cache in Algorithm 3 remains valid after goal-frontier updates. A redundancy pointer is also safe: its referenced index lies in the already-refuted interval $[r, i)$, and domination of that candidate implies domination of the current one.

Theorem 1 (Decision equivalence). Given identical heuristic arrays, successor order, arithmetic, and OPEN tie-breaking, FAST-MVH and the reference implementation make the same OPEN insertions and extractions and return the same solutions in the same order.

Proof. Induct on extractions. Both searches begin with the same effective start expansion. Exact frontier queries give the same goal-test answer. Lemma 1 gives the same local-test answer. Lemma 2 and the redundancy argument show that Algorithm 3 skips only refuted candidates; an exact negative query returns the first survivor. Frontier updates therefore retain the same live vectors. At generation, neither the local test nor selection changes a frontier. Interchanging these tests preserves their conjunction and the selected index. Both searches insert the same nodes in the same order, completing the induction. $\square$

Under the ordered admissible-MVH assumptions of [1], this equivalence preserves completeness and Pareto optimality. It does not repair an inadmissible heuristic or an invalid heuristic order. Equality between solvers on such an input would not establish correctness.

5 Representation and Cost

5.1 Adaptive frontier indexing

Each frontier starts as a contiguous array. A sufficiently large, expensive-to-scan frontier is promoted to a k-d tree over its

last $M - 1$ coordinates. Tree nodes occupy an index-addressed pool. A node stores the component-wise lower and upper corners of its descendants. For a query q , a subtree can contain a dominator only if its lower corner is at most q . Frontier updates use upper corners to exclude subtrees containing no vector dominated by the insertion. Deleted members are marked inactive; rebuilding removes them from the representation.

The archived configuration promotes frontiers of at least eight vectors when scans average 64 comparisons per query. It rebuilds when allocated node count exceeds $1.25b + 64$, where b is the live count at the last build. These parameters affect cost, not correctness. Small frontiers retain array scans.

The first-coordinate maximum is maintained with its multiplicity. If deletion removes its last occurrence, it is recomputed from live members. A stale upper bound is safe but can cause extra fallbacks; an underestimate can discard a non-dominated path.

5.2 Why not also index every heuristic set?

A frontier-tree box test compares a point with a corner. A heuristic-tree box test asks whether a corner is dominated by the *goal frontier*. The latter is a complete frontier query, not a constant-cost comparison.

For a heuristic subtree with truncated lower and upper corners ℓ, u , an index can reject every member if $\text{Exists}(T, \text{Tr}(g) + \ell)$. It can accept every member if $\neg \text{Exists}(T, \text{Tr}(g) + u)$. Both rules are sound, but neither must decide a subtree.

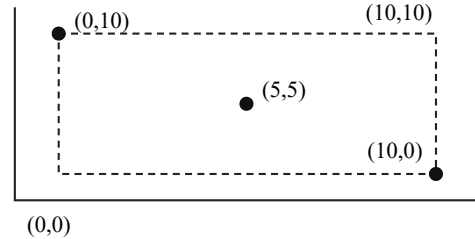


Figure 1. A heuristic box contains unattainable corners. For $g = 0$, goals at the three members refute all members but not the lower corner. A goal at $(6, 6)$ refutes the upper corner but no member.

Local-first pruning removes selection calls altogether. Witness reuse then makes many remaining refutations cheap. These changes reduce the work a static heuristic tree could save. The complete configuration evaluated here therefore leaves that optional index disabled. Cache locality is a plausible additional benefit of contiguous storage, not a measured cache-miss result.

6 Conclusion

FAST-MVH reduces selection and dominance costs without changing the search. The results favour adapting cache use as well as frontier representation. Non-lexicographic MVH search requires separate ordering arguments [5]; measurements across machines remain necessary to establish general performance.

7 Experimental Results

FAST-MVH denotes the complete cached configuration, recorded as FAST2 in the archive. The added series has 25 completed baseline-matched inputs: ten grid/heuristic combinations, two three-objective New York cases, and thirteen Bay cases. A cache-off control uses the same executable and differs only in witness caching. Harder, time-limited cases are treated separately.

7.1 Grids

The grid set separates genuine A*pex MVHs from admissible landmark families. The latter are mechanism controls: each landmark vector bounds every suffix, so their set-valued use does not demonstrate the extra pruning power of a genuine MVH. We retain their stored order and reuse existing baseline results where solution files and search counts match. New A*pex sets are lexicographically sorted for both solvers.

Table 1. Grid search times in seconds. FAST-MVH and cache-off times are minima of three interleaved repetitions. A dagger (†) marks a reused archived baseline time from an earlier session; other baselines are fresh single runs. $|\Pi|$ and expansions agree in every row. Speedup is baseline time divided by FAST-MVH time. The new 4D–6D A*pex sets use $\varepsilon = 0.05$; 3D uses the archived set. The 5D and 6D grids share the same topology and first five cost coordinates.

Grid / heuristic	M	$ \Pi $	Expansions	Baseline	FAST-MVH	Cache off	Speedup
10x10 / A*pex	3	1,602	9,499	0.0653 [†]	0.0503	0.0632	1.30
10x10 / A*pex	4	11,330	53,651	2.5541	0.4724	0.4994	5.41
8x8 / A*pex	5	9,412	33,005	1.4622	0.3091	0.3436	4.73
8x8 / A*pex	6	70,196	179,297	102.8491	2.6743	3.1317	38.46
10x10 / 50 landmarks	3	1,602	35,862	0.2087 [†]	0.1244	0.1278	1.68
15x15 / 200 landmarks	3	2,776	170,052	1.5411 [†]	0.7876	0.6909	1.96
10x10 / 50 landmarks	4	12,667	187,070	9.4335 [†]	0.9518	1.1033	9.91
12x12 / 50 landmarks	4	13,709	214,670	11.2252 [†]	1.3221	1.1437	8.49
10x10 / 50 landmarks	5	48,950	448,472	87.3698 [†]	3.3998	3.3385	25.70
10x10 / 25 landmarks	6	17,164	197,696	72.5935 [†]	2.0545	1.8457	35.33

The genuine-MVH rows establish that the improvement is not confined to landmark families or roads. On the six-objective grid, both methods return 70,196 costs after 179,297 expansions; the time difference is therefore a reduction in cost per search operation. The 4D and 5D speedups are not ordered by dimension: the instances also differ in graph size, heuristic, and frontier structure.

The cache-off column answers a separate question. On the genuine grids, the minimum-time reduction from caching is 5–20%. On several landmark controls the cached configuration is slower. This does not negate the gain over the reference search: for example, the 6D landmark control is 35.3 times faster overall while its cache-off version is faster still. The complete method and the marginal cache effect must not be conflated.

Three objectives

At $M = 3$, truncation leaves two coordinates. The three grid cases build no frontier trees. Their gains therefore come from operation order, storage, and witness handling rather than geometric indexing. The larger landmark grid also shows a cache regression of about 14%, illustrating that witness maintenance can cost more than the queries it saves.

This is not a rule to disable trees whenever $M = 3$. Bay-8 and Bay-16 build one and eleven frontier trees, respectively, and achieve 1.06 and 1.59 times the baseline speed (Table 2). The representation follows observed scan cost, not dimension alone. The smallest New York and Bay instances take only milliseconds and do not support strong relative timing claims.

7.2 Road networks

We use the DIMACS New York and San Francisco Bay Area networks [6]. Bay is also a domain in the geometric-indexing study [2]. The first two objectives are native distance and travel time; additional costs are uniform integers in $[1, 100]$, seed 42. These are synthetic added objectives, not the spatially coherent cost model of [2]. New Bay graphs retain parallel arcs and verify distance/time arc alignment. As in the baseline, search symmetrises the input graph.

NY- k and Bay- k contain k thousand vertices. They are breadth-first subgraphs, with the centre as goal and the last discovered vertex as start. Centre IDs are 140000 for NY and 160636 for Bay. The Bay sizes are 2k, 4k, 8k, and 16k. Within each Bay size, the same four extra cost fields are generated once and coordinate prefixes define 3D–6D instances. A*pex’s ε controls the heuristic, not forward-search accuracy.

Table 2. Road cases with complete baseline comparisons and nontrivial search times. New Bay rows use the timing protocol of Table 1; NY rows retain the earlier single-run comparisons. All four archived NY times measure the complete cached configuration. Times are seconds; expansions and solution costs agree within each row.

Graph	M	ε	$ \Pi $	Expansions	Baseline	FAST-MVH	Speedup
Bay-8	3	0.1	238	12,346	0.1085	0.1025	1.06
Bay-16	3	0.1	3,534	347,498	3.3258	2.0970	1.59
Bay-8	4	0.1	3,716	157,704	24.9487	1.2732	19.60
NY-5	4	0	7,105	124,837	21.858	3.236	6.8
NY-8	4	0.01	5,713	116,538	81.573	1.934	42.2
NY-5	5	0.01	26,701	396,219	326.274	9.099	35.9
NY-3	6	0.01	11,286	114,993	225.648	2.680	84.2

Table 3. Small-frontier controls retained in the evaluation. Ranges cover ten Bay cases at 2k–4k vertices: $\varepsilon = 0.1$ throughout, plus $\varepsilon = 0.05$ at Bay-2 in 4D and 5D. Two new NY-2 cases use $\varepsilon \in \{0, 0.1\}$. Search times below are milliseconds, not seconds. We do not interpret ratios at this scale; the executable reports time to 0.1 ms.

Domain	M	Cases	$ \Pi $	Baseline (ms)	FAST-MVH (ms)
Bay-2 / Bay-4	3	2	9–14	0.3–0.6	0.5–0.9
Bay-2 / Bay-4	4	3	32–37	0.5–1.1	0.8–1.8
Bay-2 / Bay-4	5	3	103–120	2.9–6.3	2.5–4.0
Bay-2 / Bay-4	6	2	157–207	5.8–9.4	5.3–7.2
NY-2	3	2	109	5.9–8.1	6.7–7.0

Table 4. Follow-up after baseline timeouts. Every baseline reached a 180 s process limit. Entries are minimum search times in seconds over three repetitions; TO denotes a 90 s process timeout and “–” means not run after that timeout. All use $\varepsilon = 0.1$. Completed cached/uncached runs agree with each other, but no baseline solution file is available.

Graph	M	Returned costs	Expansions	FAST-MVH	Cache off
Bay-8	5	33,104	1,195,876	20.7438	21.8793
Bay-8	6	–	–	377.20†	–
Bay-16	4	55,069	2,881,236	29.7028	29.9868
Bay-16	5	–	–	TO	–
Bay-16	6	–	–	TO	–

Enlarging Bay exposes denser workloads: Bay-8 in 4D promotes 210 frontiers and runs 19.6 times faster. Bay-16 in 3D returns a similar number of costs but gains only 1.59. Output size alone does not determine query cost.

Table 4 reports results from the extended timeout-fillup experiment. Bay-8 M5 (20.74s †) is the minimum of three new repetitions. Bay-8 M6 (377.20s †) completes in 6.3 minutes where baseline timed out at 180s. No baseline speedup is computed for these cases (no oracle front available).

7.3 Cache effect and operation counts

Figure 2 isolates the extra effect of goal-witness caching. For each repetition r , let a_r and b_r be cached and uncached time. The plotted saving is $100(1 - \text{median}_r(a_r/b_r))$. Thus the figure uses paired repetitions, whereas Tables 1–2 show minimum times. Differences near zero should not be read as reliable wins or losses.

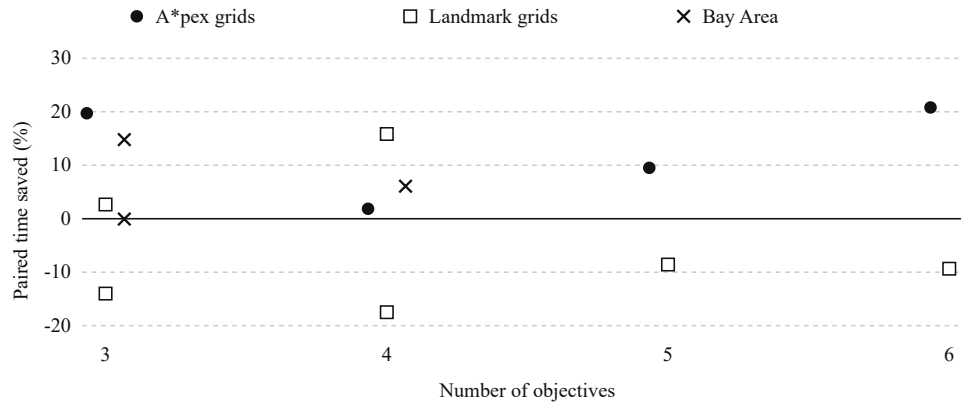


Figure 2. Marginal time saved by caching in the new complete-baseline comparisons. Positive values favour caching. For legibility, the plot omits cases whose cached or uncached minimum is below 20 ms; those cases remain in Table 3 and the result CSV. Marker offsets only separate the domain families.

Table 5. Representative operation counts for FAST-MVH. `cmpchk` and `cmpupd` are the implementation’s frontier-query and frontier-update dominance-comparison counters, in millions. They exclude separately reported heuristic and full-fallback comparisons. Fallbacks are counts, not percentages. Hits count selection queries answered by either cached witness. Trees count promotions, not rebuilds.

Case	M	<code>cmpchk</code>	<code>cmpupd</code>	Fallbacks: base / full	Cache hits	Trees
A*pex grid 10x10	3	1.13	0.17	2,468 / 3,805	81.7%	0
A*pex grid 10x10	4	9.84	2.50	12,750 / 11,777	60.0%	12
A*pex grid 8x8	5	5.05	1.24	3,608 / 3,164	69.6%	12
A*pex grid 8x8	6	32.32	7.37	14,548 / 14,804	54.8%	18
Landmark grid 15x15	3	27.57	4.28	19,498 / 2,825	11.6%	0
Landmark grid 10x10	6	35.44	8.88	105,524 / 7,573	19.7%	46
Bay-8	3	2.35	0.17	1,386 / 162	58.6%	1
Bay-8	4	50.67	6.91	94,565 / 6,414	67.1%	210

Genuine grid MVHs yield cache-hit rates of 55–82%, compared with 12–20% in the two landmark controls shown. This helps explain why a cache useful on genuine trade-offs can be overhead on a redundant family. A hit rate is not a total-time reduction: local tests, updates, queue operations, and negative queries remain. The ablation also includes indirect effects on adaptive promotion, whose observed query stream changes when cache hits bypass the frontier.

Fallback counts need not decrease. Local-first ordering tests some successors that heuristic filtering would otherwise reject before a local check. On the 3D genuine grid, fallbacks rise from 2,468 to 3,805 even though the complete search is faster. Theorem 1 preserves search decisions and OPEN operations, not the number of calls to each predicate.

The earlier same-executable New York ablation remains relevant: on five cases with four to six objectives, caching reduces minimum time by 12–38%; recorded hit rates on three cases are 77–91%. Those measurements are not combined with baseline times from different sessions. Together with the new grid regressions, they support selective rather than unconditional claims about the cache.

All new runs retain `cmpchk`, `cmpupd`, heuristic and full-dimensional comparison counts, expansions, OPEN extractions, reinsertions, fallback counts and rates, and process wall time. The unmodified baseline does not expose the same comparison counters; missing baseline counters are not treated as zero. The archive’s `gen` counter denotes OPEN extractions, not generated successors.

7.4 Protocol, validity, and limitations

Measurements use the same Intel Core i5-1135G7 laptop and Windows environment as the archive. The reused executables were built with clang/libc++ through the archived C++20 build scripts, using `-O2 -DNDEBUG -march=native`. The baseline source at commit `0a2f9ea` is unmodified. Binary hashes, source hashes, input hashes, exact arguments, affinity mask, outputs, and counters accompany each run.

New runs execute serially on one pinned logical processor. Cached and uncached order alternates between repetitions; completed comparisons have three repetitions per variant, or five for the tiny NY-2 controls. A fresh baseline runs once when no matching reference is available. Historical baseline times are marked rather than represented as contemporaneous measurements. Thermal and scheduling variation prevent confidence-interval claims from these small samples.

Search times exclude graph parsing, heuristic construction, and writing solution files. Process times and preprocessing are logged separately. Each new preprocessing or baseline process has a 180 s cap; variant runs have their own cap. A process timeout is not an exact lower bound on the internal search timer and must not be divided by search-only time to manufacture a speedup.

On all 25 completed baseline-matched inputs, all 158 cached/uncached runs reproduce the baseline solution files byte-for-byte and match expansion, extraction, and reinsertion counts. This establishes identical serialized integer costs in the same order, not identical parent paths among equal-cost alternatives. Newly built genuine MVHs are dimension-checked and lexicographically sorted. Historical landmark controls retain their order; every member independently

bounds every suffix, so arbitrary member selection remains admissible.

Some older A*pex files have first-coordinate ties that are not lexicographically ordered. Their archived agreement is empirical evidence, not a replacement for the theoretical precondition. A legacy heuristic labelled for a 6D grid was found to contain only five cost coordinates; all runs using it were excluded and replaced by newly built, dimension-matched 5D and 6D sets.

The Bay pilots and enlargements share one centre, and extra road objectives are synthetic. Instances and repetitions are not independent samples of all routing problems. The observed variation across domains, including regressions, rules out a dimension-only scaling claim. Cache misses were not measured; storage-locality explanations remain architectural rather than directly measured.

7.5 Artifacts

FAST_MVH_Expanded_Results.csv accompanies this paper and records every completed matched case, repetition count, both minimum and median timings, paired cache effect, counters, and source-run identifier. Raw runs under benchmarks/runs/ retain all repetitions and solution files, including excluded-input diagnostics and timeouts. The new run names end in `fast_mvh_grids`, `valid_grid_and_3d`, or `bay_and_3d`; the separate `2026-09-29_124302_bay_timeouts` run supplies Table 4.

The four archived NY comparisons come from `2026-09-28_175731_timeouts_fastbuild`; the five older cache ablations come from `2026-09-28_1650_tcache`. Earlier tables were consolidated in `2026-09-29_095221_paper_verification`. These remain local research artifacts, not a public replication package. No result above six objectives is included.

References

1. Wolff, M.; Felner, A.; and Salzman, O. 2026. Bridging Multi-Valued Heuristics and Dimensionality Reduction in Multi-Objective Search. *Proceedings of the Symposium on Combinatorial Search*. Reference manuscript in the project archive.
2. Anonymous. *A Geometric Index for Multi-Objective Dominance Checking*. Unpublished manuscript in the project archive, associated there with S. S. Shperberg. The supplied manuscript does not establish a public publication year or author list.
3. Geisser, F.; Haslum, P.; Thiebaux, S.; and Trevizan, F. 2022. Admissible Heuristics for Multi-Objective Planning. *Proceedings of ICAPS*, 100–109.
4. Goldberg, A. V.; and Harrelson, C. 2005. Computing the Shortest Path: A* Search Meets Graph Theory. *Proceedings of SODA*, 156–165.
5. Skyler, S.; Shperberg, S. S.; Atzmon, D.; Felner, A.; Salzman, O.; Chan, S.; Zhang, H.; Koenig, S.; Yeoh, W.; and Hernandez Ulloa, C. 2024. Theoretical Study on Multi-Objective Heuristic Search. *Proceedings of IJCAI*, 7021–7028.
6. Demetrescu, C.; Goldberg, A. V.; and Johnson, D. S., eds. 2009. *The Shortest Path Problem: Ninth DIMACS Implementation Challenge*. DIMACS Series in Discrete Mathematics and Theoretical Computer Science 74. American Mathematical Society.
7. Zhang, H.; Salzman, O.; Kumar, T. K. S.; Hernandez Ulloa, C.; Suazo, L.; and Koenig, S. 2022. A*pex: Efficient Approximate Multi-Objective Search on Graphs. *Proceedings of ICAPS*, 394–403.